Textos de Apresentação v2

Startup XYZ — Gestão de Documentos para IA | Discurso completo, alinhado à arquitetura implantada e à demo real

Equipe: Athos, Jhonatan, Gabriel, Lucas, Rafael

Esta versão substitui a v1. Ela corrige os pontos técnicos que a banca poderia contestar e troca o walkthrough simulado pela demonstração real na AWS. O texto não precisa ser decorado palavra por palavra, mas ensaiado até sair natural, sem leitura de slide.

Como usar este material

As frases em *itálico laranja* são indicações de cena: o que fazer ou apontar, e não se falam. As frases em ***destaque azul*** são os pontos de maior impacto: diga com pausa antes e depois.

Bloco 1 — Abertura, Problema e Requisitos

Athos — **3 minutos** (~302 palavras)

Slides 1 a 4 — Capa, Entendimento do Problema, Levantamento de Requisitos e Visão Geral da Solução.

[olhar para a banca, não para o slide, ao dizer o número]

Bom dia. Nós somos Athos, Gabriel, Jhonatan, Lucas e Rafael. Antes de mostrar qualquer diagrama, quero deixar um número com vocês: cinquenta mil. É a quantidade de documentos que a Startup XYZ recebe por mês, de clientes que confiam a ela arquivos que não podem ser perdidos nem vistos por outra empresa.

O problema aqui não é só guardar arquivo. É guardar o ativo que essa empresa vai usar para treinar a próxima geração da própria inteligência artificial.

A XYZ é um SaaS que extrai dados de PDFs e imagens com IA. Quando ela cresce, aparecem três problemas ao mesmo tempo. Se todo upload passar pelo servidor, o servidor vira gargalo. Com milhares de clientes, o cliente A nunca pode ver o documento do cliente B. E apagar o histórico não é opção, mas guardar tudo em armazenamento rápido faz o custo subir todo mês.

[transição de slide: Levantamento de Requisitos]

Separamos os requisitos em dois grupos. Funcionais: autenticar o usuário, receber PDF e imagem sem gargalo, isolar os documentos por cliente, permitir download seguro e preservar o histórico por anos. Não funcionais: escalar com cinquenta mil arquivos por mês, segurança e privacidade pensando na LGPD, custo proporcional ao uso, alta disponibilidade sem administrar servidores e rastreabilidade de quem acessou o quê.

Não construímos um repositório de arquivos. Construímos uma estratégia de valor ao longo do tempo.

[transição de slide: Visão Geral da Solução]

Em linguagem simples, a solução tem quatro passos. O usuário faz login e recebe um token que diz quem ele é. A API confere esse token e decide o que ele pode acessar. O sistema devolve uma chave temporária que abre só aquele arquivo, por poucos minutos. E o arquivo vai direto para o armazenamento, que depois de um ano o move sozinho para uma camada muito mais barata, sem apagar nada. O Jhonatan agora mostra como cada peça faz isso.

Bloco 2 — Arquitetura, Serviços e Justificativa

Jhonatan — **3 minutos** (~375 palavras)

Slides 5 a 8 — Diagrama de Arquitetura, Explicação dos Serviços, Justificativa da Escolha e Mapeamento de Requisitos.

[apontar o diagrama da esquerda para a direita, seguindo os números das setas]

Obrigado, Athos. A forma mais simples de resolver isso seria um servidor ligado o tempo todo esperando os uploads. Foi exatamente o que decidimos não fazer. Cinquenta mil arquivos por mês dão pouco mais de um por minuto: um servidor ligado vinte e quatro horas ficaria ocioso quase o tempo todo e ainda seria o gargalo nos picos.

[seta 1: Cliente → Cognito]

O fluxo começa no Amazon Cognito. É ele que autentica o usuário e emite um token JWT. Usamos o Cognito para não construir login do zero.

[seta 2: Cliente → API Gateway]

Com esse token, o app chama o Amazon API Gateway. O Gateway não autentica ninguém: ele valida o token do Cognito e limita o número de requisições. Sem token válido a chamada morre ali, e a Lambda nem executa.

[setas 3 e 4: Lambda e STS]

Depois vem a AWS Lambda, e aqui está o que diferencia a nossa solução. A Lambda não recebe o arquivo. Ela lê quem é o usuário no próprio token, pede ao AWS STS uma credencial temporária que só enxerga a pasta desse usuário e usa essa credencial para assinar uma URL válida por quinze minutos.

O arquivo vai direto do cliente para o Amazon S3. A Lambda nunca toca no binário, então não há limite de payload nem gargalo no backend.

[apontar para o bucket S3]

O S3 é o coração da solução. Escolhemos S3 e não EFS porque os documentos são gravados uma vez e não são editados: precisamos de durabilidade, não de sistema de arquivos. Os objetos são cifrados com uma chave nossa no AWS KMS, cada upload novo é escaneado pelo GuardDuty, e CloudWatch e CloudTrail registram o que acontece.

Uma pergunta comum: por que um bucket só, com uma pasta por cliente, e não um bucket por cliente? A AWS até permite milhares de buckets por conta, mas cada bucket seria mais uma política, mais uma regra de ciclo de vida e mais uma configuração de chave para manter. Com um bucket e uma credencial restrita à pasta, temos o mesmo isolamento com uma configuração só, e o histórico fica reunido para treinar a IA.

[transição de slide: Atendimento aos Requisitos]

Cada requisito que o Athos apresentou tem um componente concreto respondendo por ele. A tabela mostra isso linha por linha.

Agora o Gabriel mostra como essa arquitetura se comporta quando cresce, quando algo falha e quanto ela custa.

Bloco 3 — Escalabilidade, Resiliência, Segurança, Custos e Recuperação

Gabriel **3 minutos** (~397 palavras)

Slides 9 a 13 — Escalabilidade, Resiliência a Falhas, Segurança, Custos e Estratégia de Recuperação.

[pausa de um segundo antes de começar, tom mais firme]

Vou responder três perguntas antes que vocês precisem fazer: o que acontece se o volume crescer dez vezes, o que acontece se algo falhar, e quanto isso custa.

Crescimento. Cinquenta mil documentos por mês são pouco mais de um por minuto; dez vezes mais são doze por minuto. Nossa API aceita dez requisições por segundo por rota, e a conta aceita dez mil. Não existe servidor para redimensionar, e os limites que existem são cotas de conta, ajustáveis com um pedido à AWS. Na demo, cada autorização levou cerca de duzentos e cinquenta milissegundos.

Crescer dez vezes não exige redesenhar a solução. Exige, no máximo, um pedido de aumento de cota.

[transição de slide: Resiliência]

Falhas. Não há instância para cair: Lambda, API Gateway e Cognito rodam em várias zonas. O S3 grava cada objeto em pelo menos três zonas antes de confirmar o upload, por isso o RPO é praticamente zero. Se a região inteira cair, os dados continuam lá e a infraestrutura é recriada em cerca de dois minutos pelo CloudFormation. Não replicamos para outra região porque o Brasil tem uma só, e copiar dados pessoais para fora do país é transferência internacional pela LGPD.

[transição de slide: Segurança]

Segurança em três camadas. Identidade: a Lambda lê o usuário do token do Cognito, nunca do que o cliente envia. Rede: não usamos VPC de propósito, porque nenhum componente precisa de rede privada; o controle está no bucket fechado ao público, na exigência de TLS e no limite de requisições. Dados: criptografia com KMS e registro no CloudTrail de cada credencial emitida.

[apontar para a tabela de custos]

Custo. Em São Paulo, no terceiro ano, com um ano de documentos no S3 Standard e dois no Deep Archive, a conta fica em torno de sessenta dólares por mês. Sem o ciclo de vida, só o armazenamento desse histórico custaria noventa e sete.

O custo de armazenamento cai cerca de noventa e dois por cento no momento em que o documento deixa de ser operação e vira histórico.

O segundo maior item é o GuardDuty, uns vinte dólares por mês. Ele se paga porque o arquivo vem de fora, é aberto automaticamente pelo parser de IA e fica guardado para sempre. Escanear na entrada custa quatro centésimos de centavo por documento.

[transição de slide: Backup e Recuperação]

Por fim, recuperação: o versionamento desfaz qualquer sobrescrita, e a política do bucket só deixa a conta root apagar uma versão de vez. O Lucas agora mostra tudo isso funcionando.

Bloco 4 — Demonstração ao vivo

Lucas **3 minutos** (~245 palavras)

Slide 14 — Projeto em funcionamento, na conta AWS do grupo em São Paulo. Preparação no Roteiro v2.

[tela: página da demo na aba do cliente-a, já logado; console da Lambda aberto em outra aba]

Até aqui vocês viram o desenho. Agora vão ver a arquitetura rodando, na nossa conta AWS em São Paulo.

[clicar em "Chamar a API sem login"]

Primeiro, alguém de fora chama a nossa API sem login: resposta 401. Com um token forjado, 401 de novo. Isso para no API Gateway; a Lambda nem executa.

[escolher contrato-cliente-a.pdf e clicar em Enviar]

Agora o cliente A envia um contrato. A API devolveu a URL e o navegador gravou direto no S3: resposta 204. Reparem na chave: o arquivo caiu na pasta usuario- seguida do identificador desse usuário no Cognito, com um nome gerado pelo servidor.

[colar no campo 4 a chave do arquivo do cliente-b e clicar em Tentar baixar]

Agora o ataque. O cliente A descobriu o nome de um arquivo do cliente B e pede o download. A aplicação recusa: 403, chave fora do seu prefixo. Mas e se o nosso código tivesse um bug e esquecesse essa checagem?

[console: APP_CHECK = false, Salvar; voltar à página e clicar de novo em Tentar baixar]

Desliguei a checagem da aplicação para simular o bug. A Lambda assinou a URL, e o S3 respondeu 403 AccessDenied. A credencial que assinou essa URL foi emitida pelo STS só para a pasta do cliente A. O isolamento não depende de o código lembrar de checar.

[console: voltar APP_CHECK para true e salvar]

[na lista, clicar em Baixar no documento DEEP_ARCHIVE e depois em Restaurar]

Por último, o tempo. Este documento está em Deep Archive, como estaria depois de um ano. O download é recusado com a mensagem de arquivado, e o cliente pede a restauração: 202. Em até doze horas o arquivo volta. Ninguém apagou nada.

[se sobrar tempo: CloudTrail filtrado por AssumeRole]

E tudo ficou registrado: o CloudTrail mostra cada credencial emitida, com o usuário e a pasta a que ela dava acesso. Passo para o Rafael.

Bloco 5 — Evolução e Encerramento

Rafael **1,5 minuto** (~200 palavras)

Slides 15 e 16 — Evolução e Visão de Futuro, Conclusão.

[tom mais reflexivo, ritmo um pouco mais lento]

Tudo o que vocês viram resolve o problema de hoje. Quero fechar com o que vem depois.

Se a empresa crescer dez vezes, o próximo passo não é redesenhar: é colocar uma fila SQS entre o upload e a extração de IA, para que um pico de envios não sobrecarregue o processamento. A infraestrutura já é código, recriada em dois minutos pelo CloudFormation; o próximo passo é um pipeline de deploy com revisão. E, para o histórico virar base de treino com segurança, o Macie encontra dados pessoais e o treino roda num ambiente separado, com dados anonimizados.

[transição para o encerramento, olhar para todos os avaliadores]

Para fechar, volto ao número do Athos: cinquenta mil documentos por mês que não podem ser perdidos nem vistos por outro cliente, com um custo que não pode explodir com o tempo.

A resposta não é a arquitetura mais complexa. É a mais honesta com o problema: cada serviço faz o que faz de melhor, o armazenamento fica noventa e dois por cento mais barato quando o dado vira histórico, e o isolamento entre clientes foi provado ao vivo, não só no papel.

Segurança desenhada na arquitetura, custo sob controle e nenhum documento perdido. Obrigado, e ficamos à disposição para as perguntas.

Tempo total

1519 palavras de fala, cerca de 11 minutos a 140 palavras por minuto, mais as ações da demo. Fica em torno de 13 minutos, dentro da janela de 10 a 15 do guia.

O que mudou em relação à v1

- Quem autentica é o Cognito; o API Gateway só valida o token.
- O isolamento é feito por credencial temporária do STS restrita à pasta, não por "condition key de prefixo" (a condição s3:prefix só vale para listagem).
- O argumento do bucket por cliente foi atualizado: a AWS permite até 1 milhão de buckets; o motivo é operacional.
- Números em São Paulo: Deep Archive custa 8% do Standard (economia de ~92%, não 95%) e o total fica em ~US$ 60/mês no 3º ano.
- GuardDuty faz parte da arquitetura atual, com justificativa de custo; não é mais "próximo passo".
- RPO explicado como gravação em 3+ zonas (não "replicação"); falha de região e LGPD tratadas.
- O bloco do Lucas virou demonstração real, com cenas que batem com a página da demo.